

MongoDB : un écosystème en évolution

Dans un marché NoSQL très concurrentiel, MongoDB adopte une stratégie d'innovation continue :

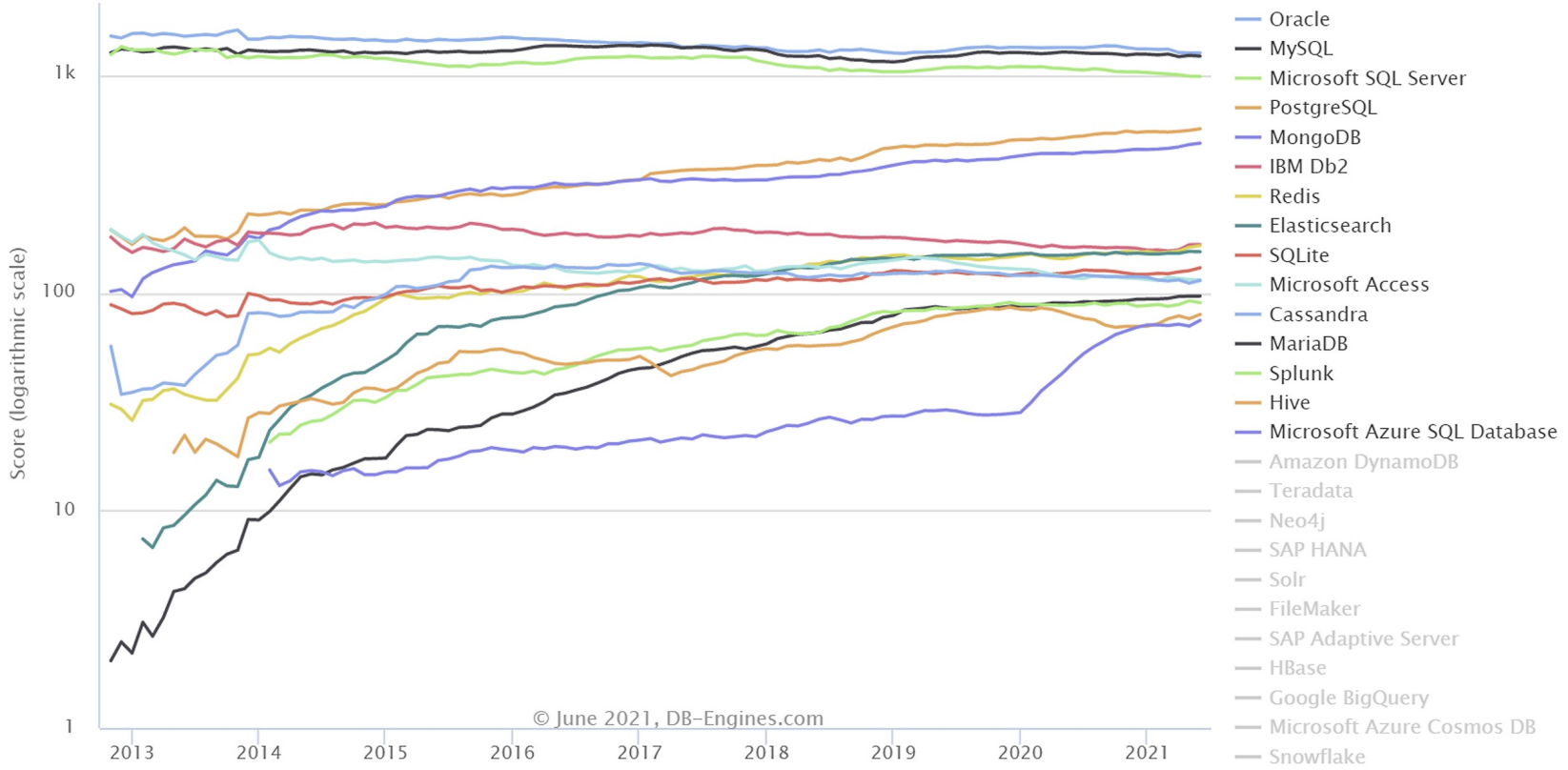
- enrichissement régulier des fonctionnalités : réplication, sharding, transactions, sécurité, recherche full-text et vectorielle ;
- outils complémentaires comme MongoDB Compass ;
- développement de MongoDB Atlas, son offre Cloud managée ;
- services et support pour les entreprises.

MongoDB évolue ainsi d'une base principalement documentaire vers une plateforme proposant des services de données de plus en plus diversifiés.

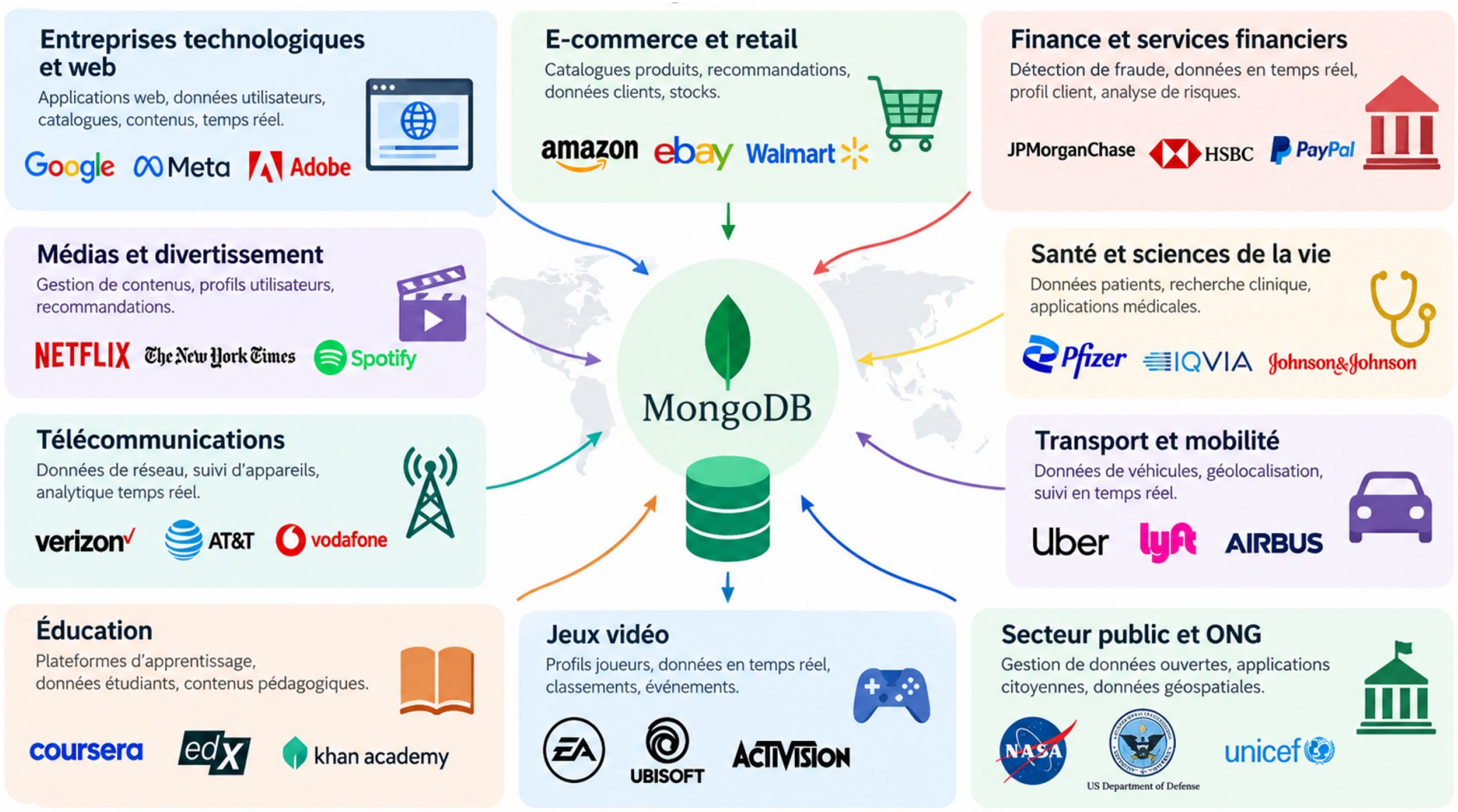
MongoDB Community est gratuit et son code source est disponible, mais les versions modernes ne sont plus considérées comme open source au sens de l'OSI. Elles sont distribuées sous licence SSPL.

MongoDB

Classement tendance (trend) des moteurs

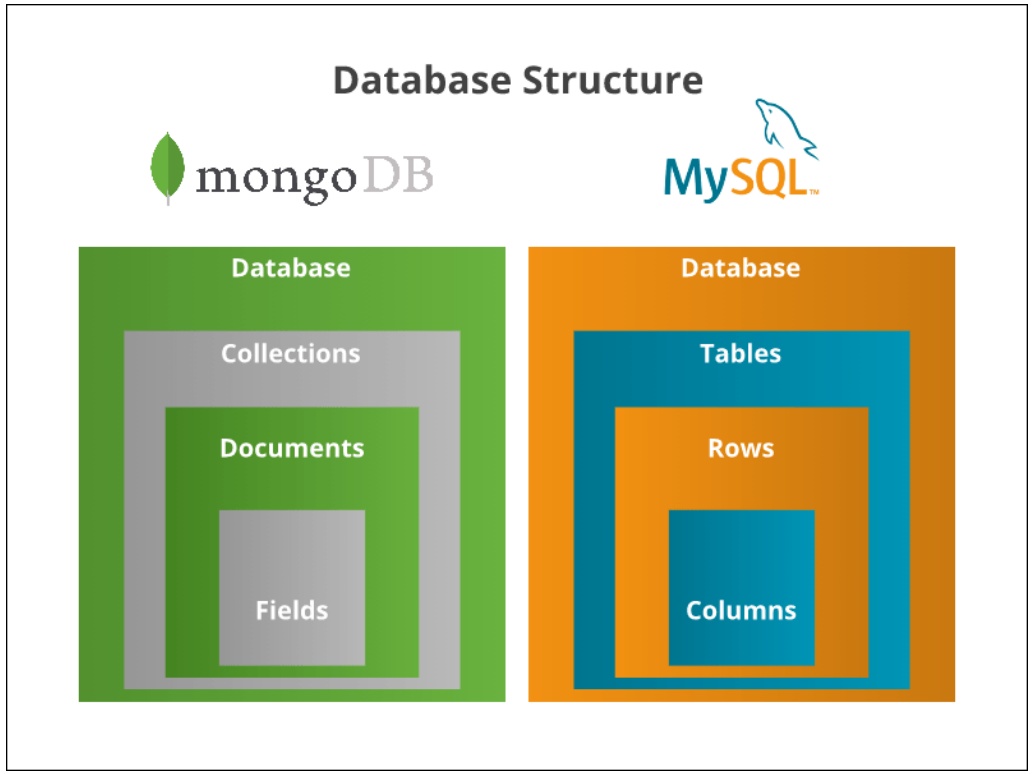


Qui utilise MongoDB ?



MongoDB


Database
Collections
Documents
Documents
Documents
Collections
Documents
Documents
Documents


Database Structure
mongoDB
MySQL
Database
Collections
Documents
Fields
Database
Tables
Rows
Columns

Structure d'un document Json

Les documents sont stockés au format BSON (Binary SON) pour optimiser le stockage et l'es échanges de documents.

Exemple de document JSON

```
{
    "id" : 123,
    "nom" : "Dupont",
    "actif" : true,
    "taille" : 1.78,
    "date_inscription" : "2025-09-20",
    "tags" : ["recherche", "données", "JSON"],
    "adresse" : {
        "rue" : "10 rue des Sciences",
        "ville" : "Paris",
        "code_postal" : "75005"
    },
    "projets" : [
        {
            "nom" : "rBDD",
            "role" : "membre"
        },
        {   "nom" : "OpenData",
            "role" : "contributeur"
        }
    ]
}
```

Types scalaires (valeurs simples)

Les types scalaires représentent une seule valeur.

Chaîne de caractères (string)	`"Dupont"`	Entre guillemets
Nombre entier (number)	`123`	Sans guillemets
Nombre décimal (number)	`1.78`	Avec un séparateur .
Booléen (boolean)	`true`	true ou false
Valeur nulle (null)	`null`	Représente l'absence de valeur

Tableau (array)

Un tableau est une liste ordonnée de valeurs, qui peuvent être des scalaires, des documents ou un mélange des deux.

```
["recherche", "données", "JSON"]
```

Tableau de chaînes de caractères

Document (objet)

Un document (ou objet) est un ensemble de paires clé / valeur.

```
{
   "rue" : "10 rue des Sciences",
   "ville" : "Paris",
   "code_postal" : "75005"
}
```

Les valeurs d'un document peuvent elles-mêmes être des scalaires, des tableaux ou d'autres documents.

Tableau de documents

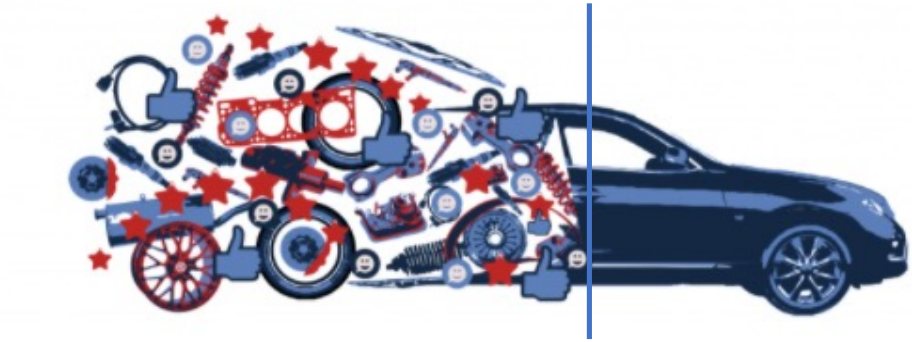
Un tableau peut contenir plusieurs documents.

```
[
   { "nom" : "rBDD", "role" : "membre"  },
   { "nom" : "OpenData", "role" : "contributeur" }
]
```

Modèle relationnel vs modèle Document

Introduction au modèle orienté document

Modèle de données Relationnel



itemid	orderid	item	amount
5	1	Chair	200.00
6	1	Table	200.00
7	1	Lamp	123.12

customerid	name	email
5	Rosalyn Rivera	rosalyn@adatum.com
6	Jayne Sargent	jayne@contoso.com
7	Dean Luong	dean@contoso.com

orderid	customerid	date	amount
1	4	11/1/17	523.12
2	3	11/15/17	32.99
3	1	11/21/17	23.99

Représentation logique sous forme de tables à 2 dimensions

Modèle de donnée orienté Document



Représentation logique sous forme de documents

Modèle relationnel vs modèle Document

Modèle relationnel

Des données réparties dans plusieurs tables

Les informations d'un même objet métier sont stockées dans plusieurs tables et liées par des clés.
Pour reconstruire l'objet complet, il faut effectuer des jointures.

CLIENT

id	nom	ville
1	Dupont	Lyon
2	Martin	Paris
3	Bernard	Toulouse

COMMANDE

id	client_id	date
1001	1	2025-09-01
1002	1	2025-09-12
1003	2	2025-09-15

LIGNE_COMMANDE

id	commande_id	produit_id	quantite
1	1001	10	1
2	1001	11	2
3	1002	10	1
4	1003	12	1

PRODUIT

id	nom	prix
10	Clavier	80
11	Souris	30
12	Ecran	200

Pour obtenir une commande complète :

- Plusieurs jointures sont nécessaires
- Les données sont réparties dans différentes tables

Avantages du modèle relationnel

Intégrité des données
Contraintes (clés, unicité, référentielle).

Transactions ACID
Garantit la cohérence même en cas d'erreur ou de concurrence.

Requêtes complexes
Jointures, agrégations, analyses fines.

Maturité et écosystème riche
Outils, normes, retours d'expérience.

Inconvénients

Schéma rigide
Évolution plus complexe.

Multiples jointures
Peut impacter les performances et la scalabilité.

Distribution plus complexe
Nécessite des mécanismes sophistiqués pour répartir les données.

Moins adapté aux données dynamiques ou très variées.

Modèle relationnel vs modèle Document

2 façons d'organiser les données, 2 approches pour des besoins différents

Modèle relationnel

Des données réparties dans plusieurs tables

Les informations d'un même objet métier sont stockées dans plusieurs tables et liées par des clés.
Pour reconstruire l'objet complet, il faut effectuer des jointures.

CLIENT

id	nom	ville
1	Dupont	Lyon
2	Martin	Paris
3	Bernard	Toulouse

1 — N

COMMANDE

id	client_id	date
1001	1	2025-09-01
1002	1	2025-09-12
1003	2	2025-09-15

1 — N

LIGNE_COMMANDE

id	commande_id	produit_id	quantite
1	1001	10	1
2	1001	11	2
3	1002	10	1
4	1003	12	1

N — 1

PRODUIT

id	nom	prix
10	Clavier	80
11	Souris	30
12	Ecran	200

Pour obtenir une commande complète :

- Plusieurs jointures sont nécessaires
- Les données sont réparties dans différentes tables



Même information mais une organisation différente

Modèle document

Un document auto-suffisant

Les **données fréquemment** utilisées ensemble sont regroupées dans un même document (dénormalisation).
Une seule lecture permet de récupérer l'objet complet, sans jointure.

Exemple de document JSON (une commande)

```
{
  "id": 1001,
  "client": {
    "id": 1,
    "nom": "Dupont",
    "ville": "Lyon"
  },
  "date": "2025-09-01",
  "produits": [
    { "id": 10, "nom": "Clavier", "prix": 80, "quantite": 1 },
    { "id": 11, "nom": "Souris", "prix": 30, "quantite": 2 }
  ],
  "total": 140
}
```

Un document autonome

- Toutes les informations utiles sont contenues dans le document
- Une seule lecture suffit
- Peu ou pas de jointures dans les cas d'usage courants

Modèle relationnel vs modèle Document

Modèle document

Un document auto-suffisant

Les **données fréquemment** utilisées ensemble sont regroupées dans un même document (dénormalisation).
Une seule lecture permet de récupérer l'objet complet, sans jointure.

Exemple de document JSON (une commande)

```
{
  "id": 1001,
  "client": {
    "id": 1,
    "nom": "Dupont",
    "ville": "Lyon"
  },
  "date": "2025-09-01",
  "produits": [
    { "id": 10, "nom": "Clavier", "prix": 80, "quantite": 1 },
    { "id": 11, "nom": "Souris", "prix": 30, "quantite": 2 }
  ],
  "total": 140
}
```

Un document autonome

- ✔ Toutes les informations utiles sont contenues dans le document
- ✔ Une seule lecture suffit
- ✔ Peu ou pas de jointures dans les cas d'usage courants

Avantages du modèle document

Passage à l'échelle facilité
Documents autonomes, faciles à distribuer (sharding).

Modèle flexible
Pas de schéma fixe, s'adapte aux évolutions.

Moins (ou pas) de jointures
Une lecture suffit pour accéder à l'objet complet.

Inconvénients

Pas (ou peu) transactionnel
Pas de transactions ACID multi-documents (nativement).
Les transactions existent mais sont limitées (en périmètre et en performances).

Risque de duplication
Les données peuvent être dupliquées entre documents (cohérence à gérer).

Moins adapté à certaines contraintes fortes d'intégrité et de cohérence globale.

Modèle relationnel vs modèle Document

Distribution des données

Dans une base relationnelle

Les données liées sont réparties dans différentes tables, ce qui complique la distribution et peut nécessiter des échanges entre machines pour réaliser les jointures.



Dans une base orientée documents

Chaque document est une unité de données autonome qui peut être stockée sur n'importe quelle machine. Les requêtes sont majoritairement locales.



En regroupant les données liées dans un même document :
- on limite les jointures
- on favorise la localité des données
- on facilite la distribution à grande échelle.


CREATE
READ
UPDATE
DELETE
C R U D

CRUD : Create

Comment créer un document dans une base mongo ?

```
db.products.insertOne( { item: "card", qty: 15 } )
```

db : objet de connexion à la bdd
Product : nom de la collection
insertOne : action d'insertion d'un seul document



MongoDB collection **products**

```
{ "_id" : ObjectId("5063114bd386d8fadbd6b004"), "item" : "card", "qty" : 15 }
```

Chaque document possède un identifiant unique (_id).
S'il n'est pas précisé lors de l'insertion du document, MongoDB le génère automatiquement.

CRUD : Create

```
db.products.insertMany(
  [
    { item: "pen", qty: 20 },
    { _id: 11, item: "pencil", qty: 50, type: "no.2" },
    { item: "eraser", qty: 25 }
  ]
)
```



```
{ "_id" : ObjectId("51e0373c6f35bd826f47e9a0"), "item" : "pen", "qty" : 20 }
{ "_id" : 11, "item" : "pencil", "qty" : 50, "type" : "no.2" }
{ "_id" : ObjectId("51e0373c6f35bd826f47e9a1"), "item" : "eraser", "qty" : 25 }
```

Lors d'une insertion multiple, si une erreur survient lors de l'insertion d'un document, MongoDB conserve les documents déjà insérés : il n'y a pas de rollback automatique de l'ensemble de l'opération.

CRUD : Read

db.collection.find({condition},{projection}) return cursor

La condition est exprimée sous forme de document : { "champ" : valeur , … }

Rechercher tous les restaurants dans le quartier (borough) de Brooklyn:

db.restaurants.find({ "borough" : "Brooklyn" })

Rechercher les restaurants dans le quartier (borough) de Brooklyn **ET** dont la cuisine est italienne.

db.restaurants.find({ "borough" : "Brooklyn", "cuisine" : "Italian" })

CRUD : Read

Le **2e paramètre de find()** permet de sélectionner les champs à retourner.

```
db.collection.find(
  { filtre },
  { champ1: 1, champ2: 1, _id: 0 }
)
```

```
Champ1 : 1  / true → le résultat affiche le champ champ1
_id : 0  / false → le résultat n'affiche pas le champ _id
```

```
db.restaurants.find(
        { "borough" : "Brooklyn", "cuisine" : "Italian" } ,
        {borough:1}
)
```

Affiche uniquement le champs borough de chaque document : par défaut _id est affiché.

```
db.restaurants.find(
        { "borough" : "Brooklyn", "cuisine" : "Italian" } ,
        {borough:0}
)
```

Affiche tout les champs du document satifaisant la condition à l'exception du champs borough (borough:0)

CRUD : Read

```
{
  "_id" : 1,
  "name" : "sue",
  "age" : 19,
  "type" : 1,
  "status" : "P",
  "favorites" : { "artist" : "Picasso", "food" : "pizza" },
  "finished" : [ 17, 3 ]
  "badges" : [ "blue", "black" ],
  "points" : [ { "points" : 85, "bonus" : 20 }, { "points" : 85, "bonus" : 10 } ]
}
```

Quelle requête retourne le document ci-dessus :

- db.users.find({ badges: "black" },{ name: 1})
- db.users.find({ "badges.0": "black", { points: 0 })
- db.users.find({ "favorites.food": "Picasso" }, {"points.0.bonus": 0)
- db.users.find({ statu: "A", age: { $lt: 30 } }).skip(0).limit(5)

CRUD : Read

```
db.addressBook.insertMany(
  [
    { "_id" : 1, address : "2030 Martian Way", zipCode : "90698345","contact" : true },
    { "_id" : 2, address: "156 Lunar Place", zipCode : 43339374,"contact" : false },
    { "_id" : 3, address : "2324 Pluto Place", zipCode: NumberLong(3921412),"contact" : false },
    { "_id" : 4, address : "55 Saturn Ring" , zipCode : NumberInt(88602117) },
    { "_id" : 5, address : "104 Venus Drive", zipCode : ["834847278", "1893289032"],"contact" : true },
    { "_id" : 6, address : "222 Sun Boulevard", zipCode : null,"contact" : true }
  ]
)
```

```
db.addressBook.find( { "zipCode" : { $type : "string" } } )              => {1,5}

db.addressBook.find( { "zipCode" : {$not : { $type : "null" } } } )   => {1,2,3,4,5}

db.addressBook.find( { "contact" : { $exists : true} } )                 => {1,2,3,5,6}

db.addressBook.find( { "contact" : { $exists : true, $eq : true} } )  => {1,5,6}
```

CRUD : Read

- Opérateurs de comparaisons : $eq, $gt, $gte, $in $nin
  db.inventory.find({ qty: { $nin: [5, 15] } })
  https://docs.mongodb.com/manual/reference/operator/query-comparison/

- Opérateurs logiques : $and, $or, $not, $nor
  db.inventory.find({ price: { $not: { $gt: 1.99 } } })
  https://docs.mongodb.com/manual/reference/operator/query-logical/

- Opérateur sur les types de valeur : $exists, $type
  db.inventory.find({ price: { $type : “double” } })
  https://docs.mongodb.com/manual/reference/operator/query-element/

- Opérateur sur les tableaux
  https://www.mongodb.com/docs/manual/reference/mql/query-predicates/arrays/

CRUD : Read

Une requête peut retourner **des centaines, des milliers, voire des millions de documents**.

MongoDB ne retourne pas nécessairement tous les résultats en une seule fois. find() retourne un **curseur**, qui permet de parcourir les résultats **progressivement, par lots (batches)**.

Cela permet notamment de :

- limiter la **mémoire utilisée** ;
- limiter les **transferts réseau** ;
- traiter progressivement de grands volumes de résultats.

- Exemple :
  - limit(): contraint la taille de l'ensemble résultat
  - count(): modifie le curseur pour retourner le nombre de documents plutôt que les documents eux-mêmes.
  - skip(): retourne un curseur qui pointe sur les documents résultat après avoir passé le nombre de documents indiqué.
  - pretty(): configure le curseur pour afficher les documents de manière plus facile à lire (pour un humain)

```
db.trips.find({"usertype" : "Customer"}).count()
db.zips.find({"state" :"CA"}).skip(500).limit(10)
```

La liste complète :
https://docs.mongodb.com/manual/reference/method/js-cursor/

CRUD : Update

- MongoDB propose plusieurs méthodes de mise à jour :

Db.mycollection.updateOne({$query},{$set}) / modifie **un seul document**, même si plusieurs correspondent au filtre.

Db.mycollection.updateMany({$query},{$set}) → modifie **tous les documents** correspondant au filtre.

db.collection.replaceOne({$query},{replacement}) → **remplace entièrement** un document.

```
db.users.updateOne(
  { ville: "Lyon" },
  { $set: { actif: true } }
)
```

```
db.restaurant.replaceOne(
    { "name" : "Central Perk Cafe" },
    { "name" : "Central Pork Cafe", "Borough" : "Manhattan" }
  );
```

CRUD : Update



- Une opération portant sur un seul document est atomique.

- Une opération updateMany() est atomique **document par document**, mais l'ensemble de l'opération n'est pas une transaction atomique.

- Pour garantir une modification **tout ou rien** sur plusieurs documents, il faut utiliser une **transaction multi-documents**.

Delete

db.collection.deleteOne(filtre)
-> Supprime **un seul document** vérifiant le filtre. Si plusieurs documents correspondent, **un seul est supprimé**.

db.collection.deleteMany(filtre)
-> Supprime **tous les documents** correspondant au filtre.

```
db.products.deleteOne({
  item: "pencil",
  type: "blue"
})
```



```
{ "acknowledged": true, "deletedCount": 1 }
```

```
db.products.deleteMany({
   type: "blue"
})
```



```
{ "acknowledged": true, "deletedCount": 5 }
```

TP MONGO CRUD



Aggregation framework

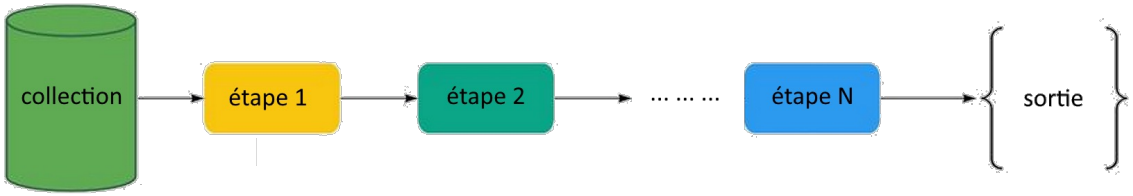


Aggregation framework

MongoDB intègre une plateforme qui permet d'enchâiner un ensemble d'opération l'une à la suite des autres.

Le pipeline d'agrégation est :

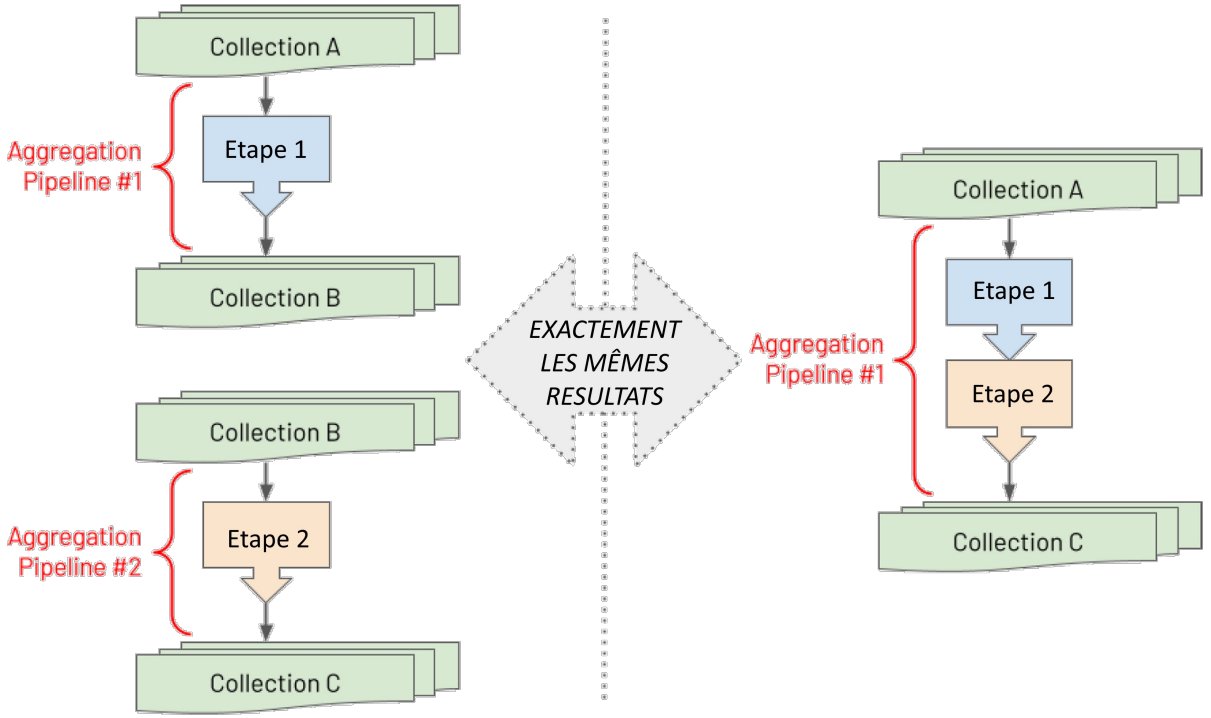
- - une série ordonnée d'opérations déclaratives appelées étapes
- - chaque étape réalise une transformation sur les documents en entrée
- - une étape peut exclure, synthétiser ou créer des documents
- - la totalité de la sortie d'une étape constitue l'entrée de l'étape suivante
- - la sortie finale est un curseur, une (nouvelle) collection ou une vue



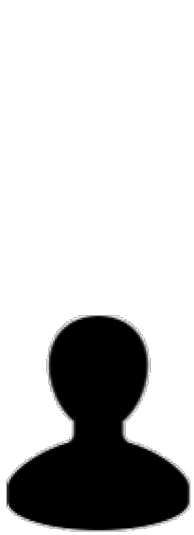
Aggregation framework

Les étages sont autonomes et sans état

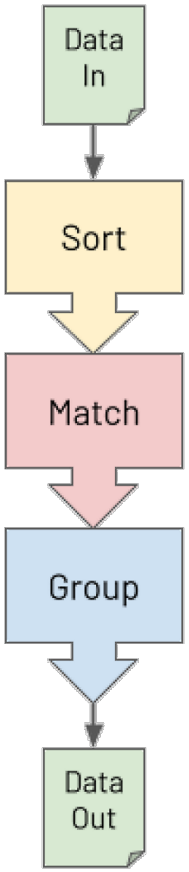
Avec l'agrégation, vous pouvez prendre un problème complexe demandant une agrégation complexe et le décomposer en série d'étapes plus simples qui peuvent être testées et validées indépendamment.

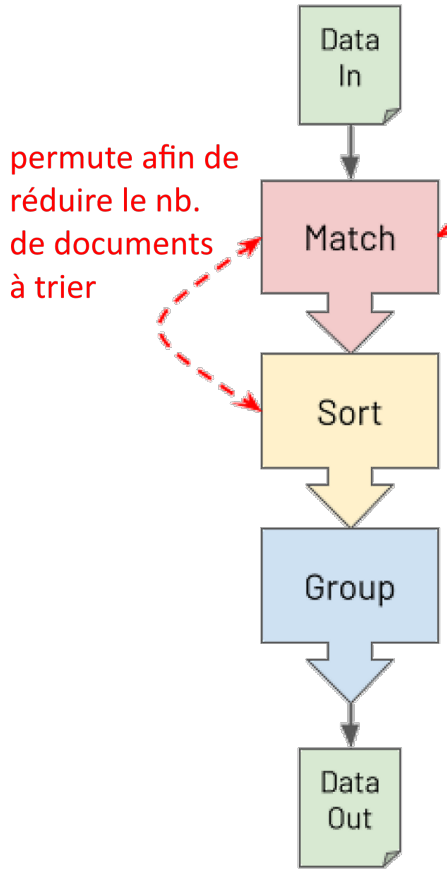


Aggregation framework



Développeur





permute afin de réduire le nb. de documents à trier

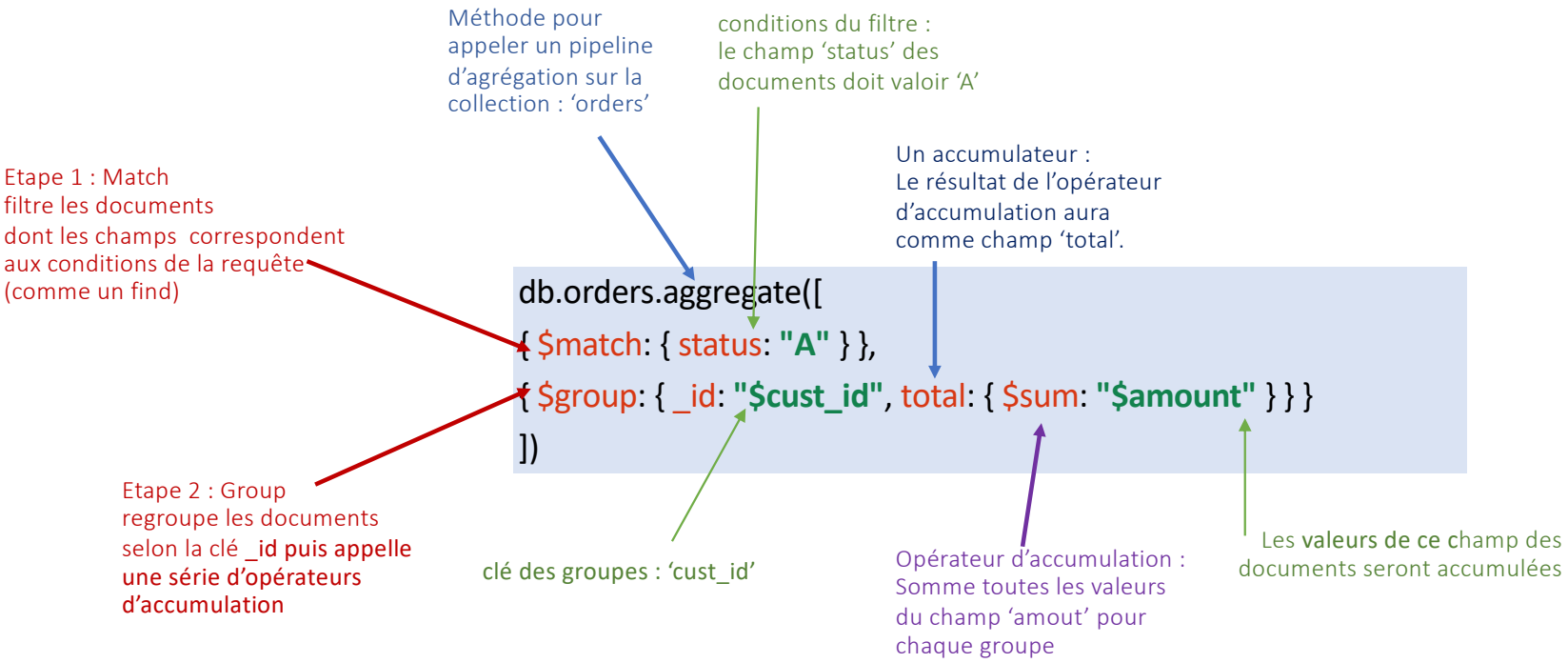
cible les index pour le filtre de "Match"

lance un "match" en // sur chaque "shard" pour des sous-ensembles

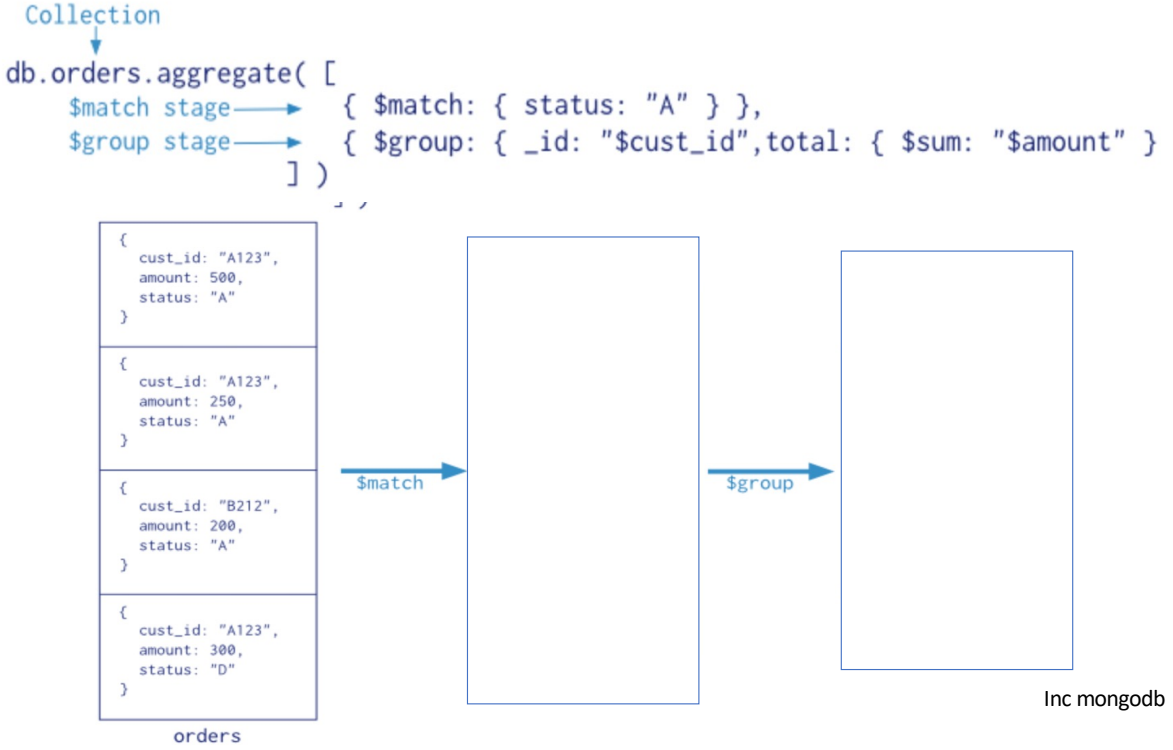


Moteur de bases de données

Aggregation framework

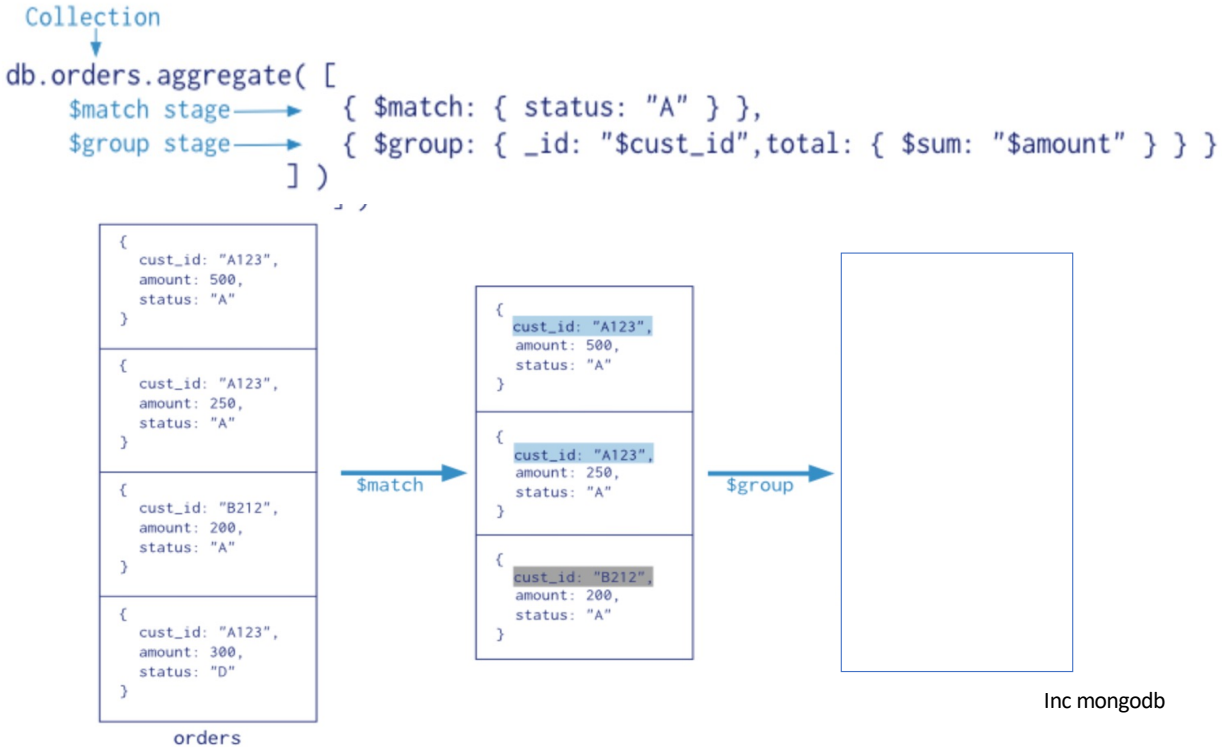


Aggregation framework



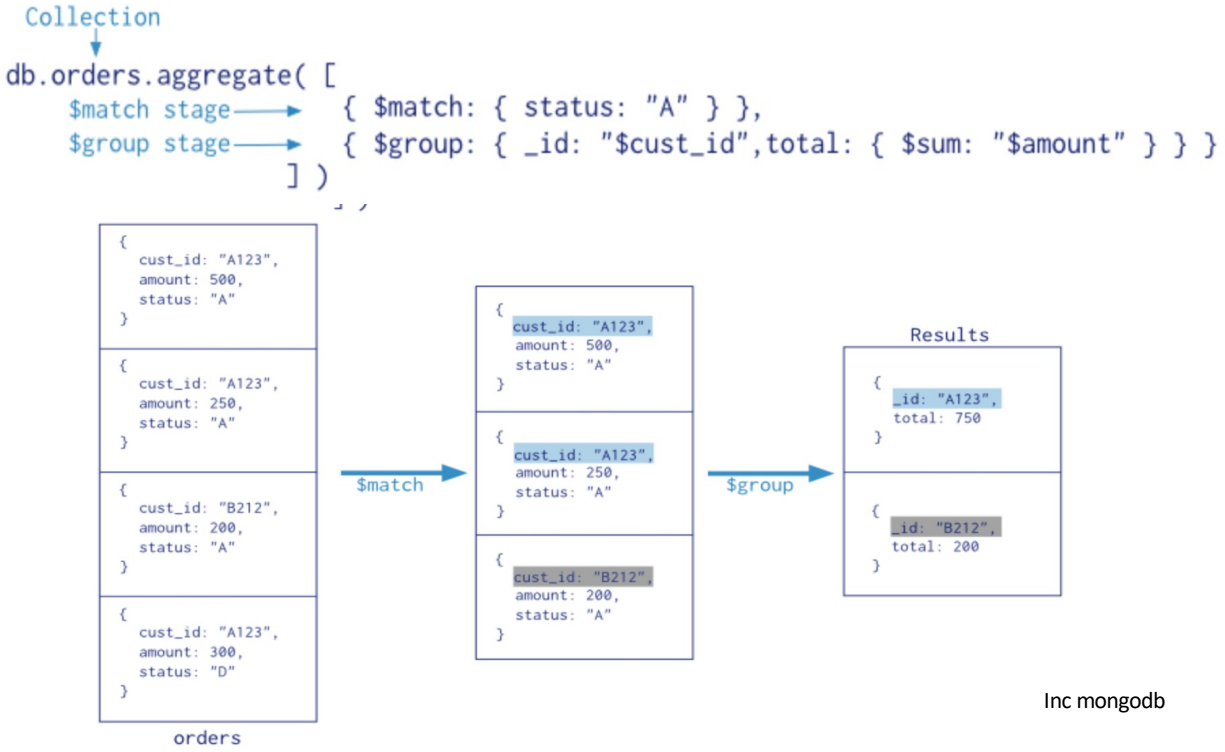
Inc mongodb

Aggregation framework



Inc mongodb

Aggregation framework



Inc mongodb

Aggregation framework

$match : filtre les documents
$project : redéfinition des champs de projection
$lookup : jointure gauche
$group : regroupe les données par valeur
…

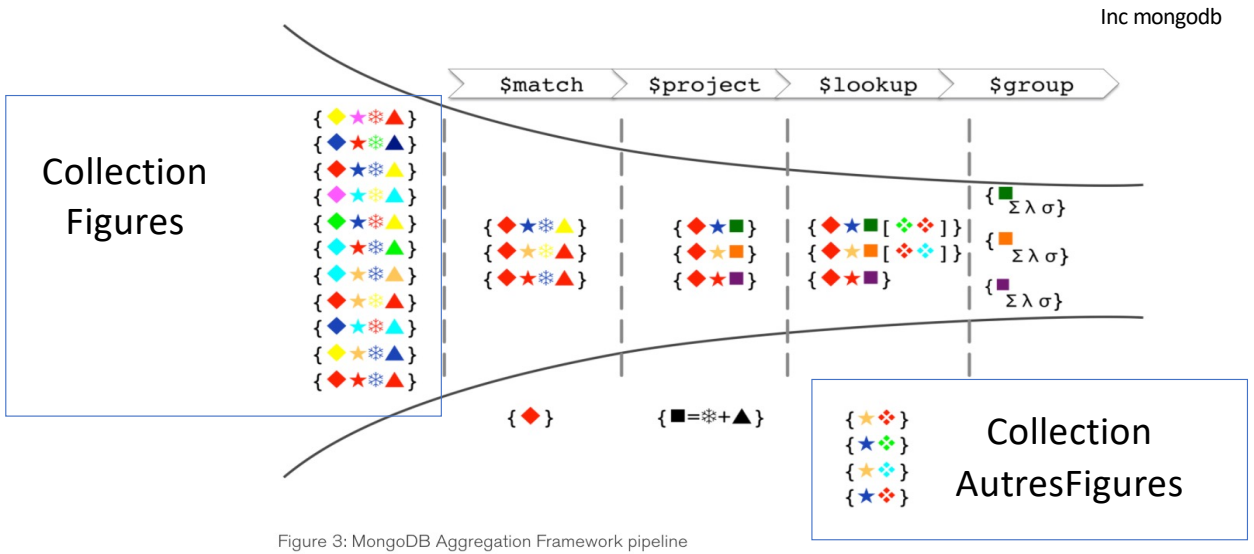


Figure 3: MongoDB Aggregation Framework pipeline

Aggregation framework



Les étapes peuvent servir à filtrer, projeter, trier, formater, … les documents au fur et à mesure de leur avancement dans le pipeline.

La liste complète peut être retrouvée ici :

- https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/

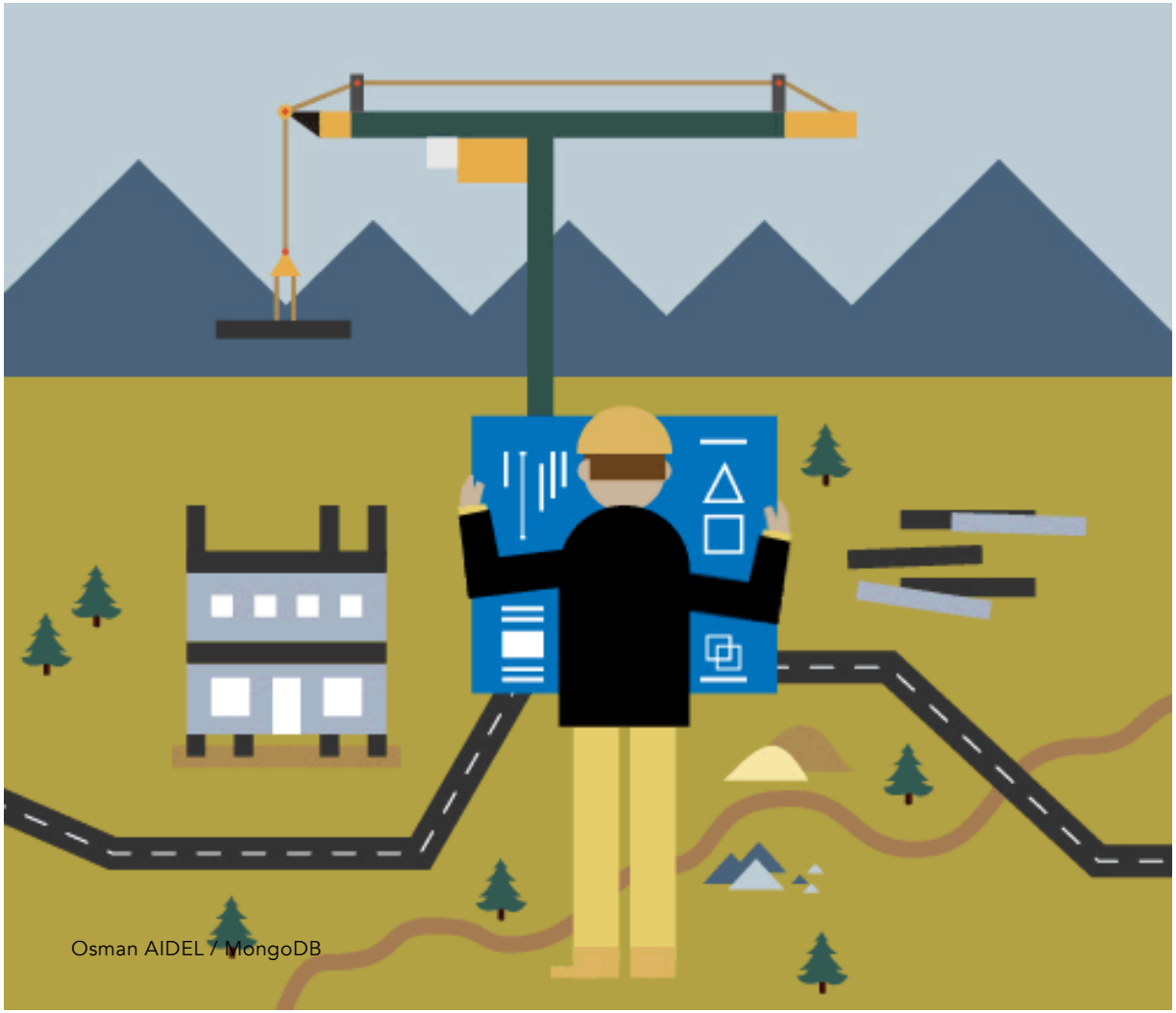
Les étapes utilisent des opérateurs pour réaliser les opérations.
Ils sont au nombre de 135 et, comme les étapes peuvent être de différents types :

- Les opérateurs arithmétiques / Les opérateurs sur les tableaux / Les opérateurs booléens
- Les opérateurs sur les tailles / Les opérateurs sur les types / Les opérateurs sur les dates
- Les opérateurs sur les objets / Les opérateurs sur les chaînes / Les opérateurs trigonométriques

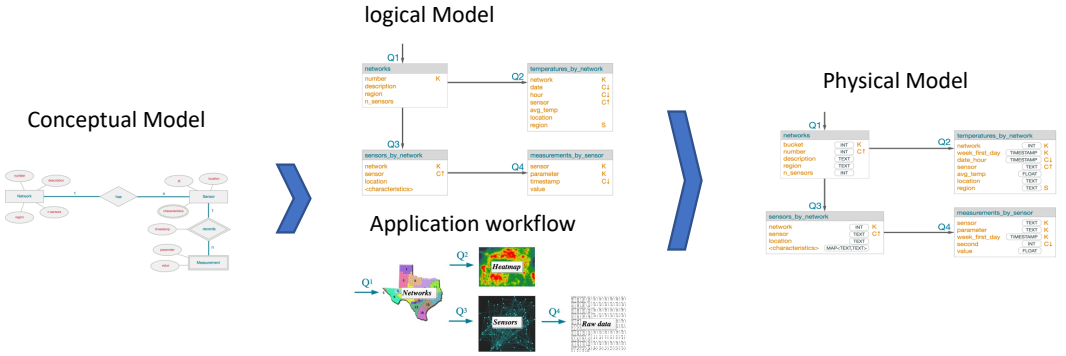
TP



Introduction au modèle orienté document



Introduction au modèle orienté document



Dans les bases de données relationnelles, le métier guide le modèle de données et est complétement dé-corrélé de la manière dont les données seront accédées par l'application.

Si les performances ne sont pas au rendez-vous, on ajoute des structures complémentaires pour palier à ce problème ce qui ne suffit pas tout le temps surtout dans un environnement BIG DATA.

Dans le monde NoSQL, le modèle de donnée prend en compte dès la conception du modèle la manière dont les accès seront réalisés. Cette approche confère au modèle NoSQL :

- La performance est privilégiée.
- Le modèle physique est construit à partir du modèle conceptuel et de la nature des accès aux données (INSERT, SELECT, UPDATE …).
- Le modèle physique est organisée de manière optimale pour l'application. Toutefois, la normalisation des données est souvent incompatible avec cette approche et les données sont souvent dupliquées.
- Gagner en performance au détriment de la cohérence : Approche Query-Driven

L'approche Query-Driven privilégie l'organisation des données selon les besoins d'accès de l'application. Elle peut conduire à dénormaliser certaines informations pour améliorer les performances, au prix d'une complexité supplémentaire dans le maintien de leur cohérence.

Introduction au modèle orienté document

Pour modéliser vos données, il est important d'identifier les points suivants :

Les scenarii

- Décrire les enchainements des requêtes.

La nature du jeux de données

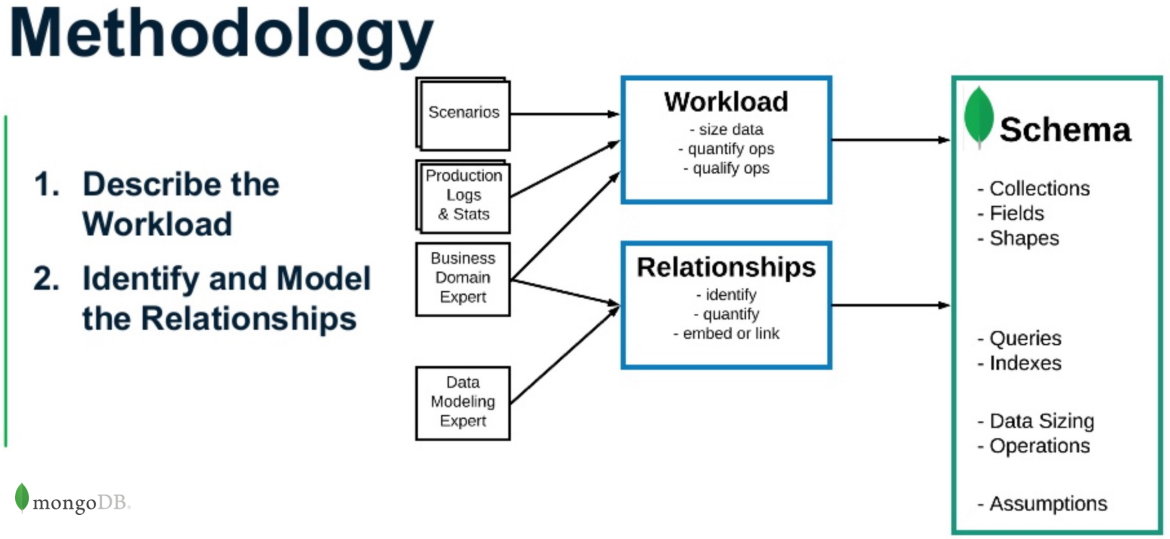
- La taille : BIG DATA
- Données en temps réel
- Données analytiques

La nature des accès

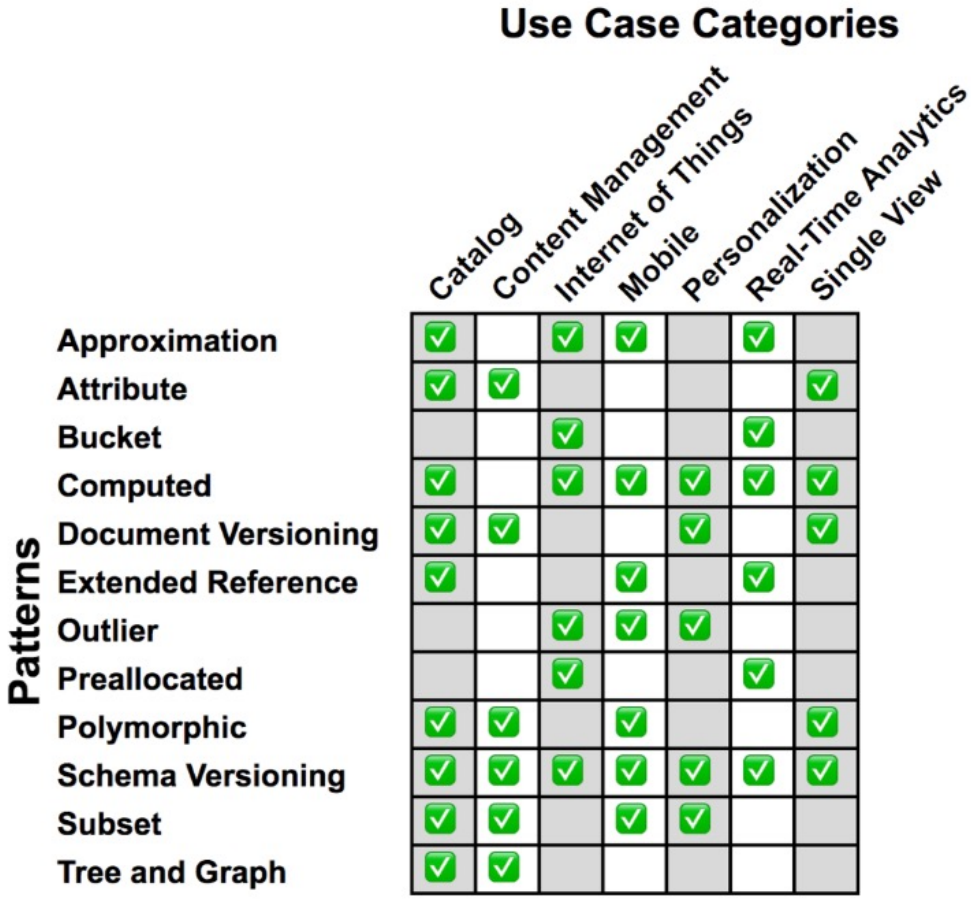
- Quantifier et qualifier les opérations
  - fréquence
  - Taille
  - Read / Write

Les contraintes métiers

- Identifier et quantifier les relations
- Fraicheur des données.
- Les critères de performance



Design Patterns



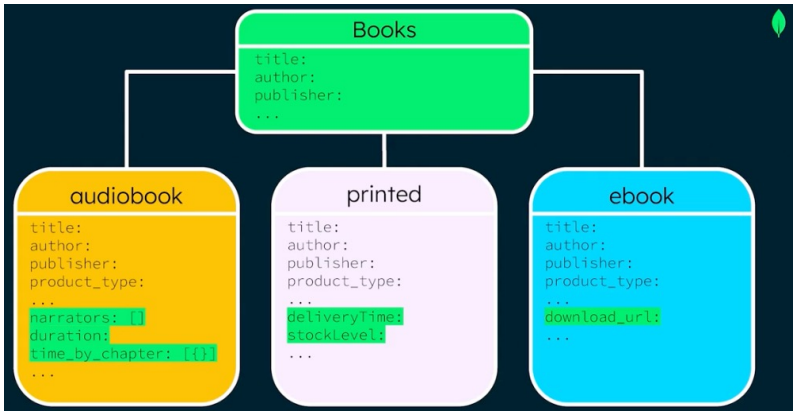
Pattern d'héritage / Polymorphisme

Ce modèle est le point de départ des modèles de conception sous MongoDB.

Ce modèle est utilisé lorsque tous **les documents d'une collection ont une structure similaire mais pas identique.**

- Il y a plus de similarité que de différences entre les documents

Il est utile lorsqu'on souhaite accéder à des informations à partir d'une seule collection, ce qui améliore les performances en évitant de séparer les objets dans plusieurs collections.



Les données qui sont accédées ensembles doivent être stockées ensembles :

quelque soit le support du livre nous devons les stockées ensembles.

Pattern d'héritage / Polymorphisme

printedBooks

```
{
  "_id": 1,
  "product_id": 34538756,
  "title": "MongoDB: The Definitive Guide: Powerful and Scalable Data Storage",
  "details": "MongoDB explained by MongoDB champions",
  "authors": [ "Shannon Bradshaw", "Eoin Brazil", "Christina Chodorow" ],
  "publisher": "O'Reilly",
  "language": "English",
  "pages": 514,
  "catalogues": { "isbn10": "1491954469", "isbn13": "978-1491954461" }
}
```

eBooks

```
{
  "_id": 2,
  "title": "MongoDB: The Definitive Guide: Powerful and Scalable Data Storage",
  "desc": "MongoDB explained by MongoDB champions",
  "authors": "Shannon Bradshaw, Eoin Brazil, and Christina Chodorow",
  "publisher": "O'Reilly",
  "language": "English",
  "eformats": { "epub": { "pages": 774 }, "pdf": { "pages": 502 } },
  "isbn10": "1491954469"
}
```

audioBooks

```
{
  "_id": 3,
  "product_id": 54538756,
  "title": "MongoDB: The Definitive Guide: Powerful and Scalable Data Storage",
  "desc": "The complete book of MongoDB by its employees",
  "author": "Eoin Brazil",
  "narrator": "Eoin Brazil",
  "publisher": "O'Reilly",
  "language": "English",
  "length_minutes": 1200
}
```

Aggregation framework

- Fusionner les documents ($merge)
- Uniformiser les champs communs ($set,$project,$unset)
- Ajouter les champs product_type et product_id ($set)

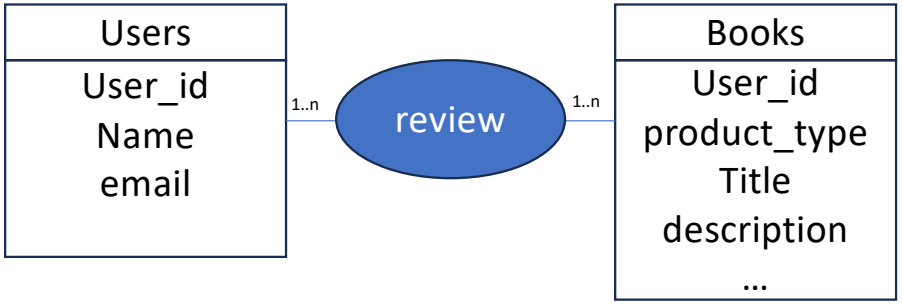
Books collection

```
{
  "_id": 1,
  "product_id": 34538756,
  "product_type": "book",
  "title": "MongoDB: The Definitive Guide: Powerful and Scalable Data Storage",
  "description": "MongoDB explained by MongoDB champions",
  "authors": [ "Shannon Bradshaw", "Eoin Brazil", "Christina Chodorow" ],
  "publisher": "O'Reilly",
  "language": "English",
  "pages": 514,
  "catalogues": { "isbn10": "1491954469", "isbn13": "978-1491954461" }
}
```

```
{
  "_id": 2,
  "product_id": 44538756,
  "product_type": "ebook",
  "title": "MongoDB: The Definitive Guide: Powerful and Scalable Data Storage",
  "description": "MongoDB explained by MongoDB champions",
  "authors": [ "Shannon Bradshaw", "Eoin Brazil", "Christina Chodorow" ],
  "publisher": "O'Reilly",
  "language": "English",
  "eformats": { "epub": { "pages": 774 }, "pdf": { "pages": 502 } },
  "isbn10": "1491954469"
}
```

```
db.books_bdf78916_83ea_11ef_a3e7_0202c8411e48.replaceOne({ _id: 3 }, {
  "_id": 3,
  "product_id": 54538756,
  "product_type": "audiobook",
  "title": "MongoDB: The Definitive Guide: Powerful and Scalable Data Storage",
  "description": "The complete book of MongoDB by its employees",
  "authors": [ "Eoin Brazil" ],
  "narrator": "Eoin Brazil",
  "publisher": "O'Reilly",
  "language": "English",
  "length_minutes": 1200
}
```

Subset Pattern



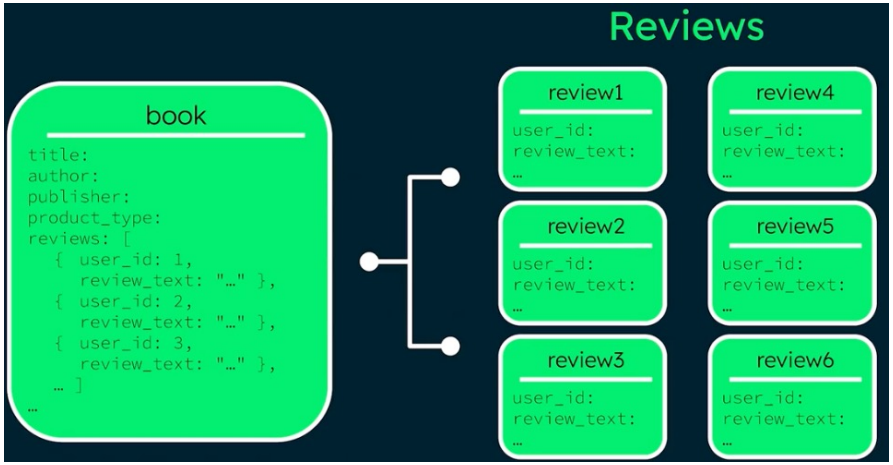


Une application gère les critiques sur des livres.

Si le nombre de reviews par livre est très important l'imbrication des reviews ne sera pas possible à cause de la taille d'un document limité à 16Mo.

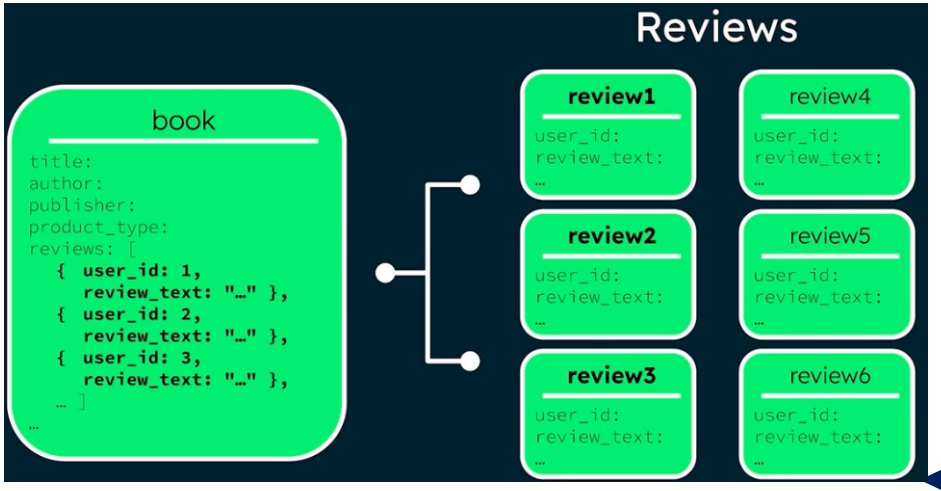
```json
{
  "_id": "book_id_1",
  "title": "Mastering MongoDB",
  "author": "John Doe",
  "publisher": "MongoDB Press",
  "language": "English",
  "pages": 300,
  "reviews": [
    {
      "review_id": "review_1",
      "user": {
        "user_id": "user_1",
        "name": "Alice",
        "email": "alice@example.com"
      },
      "rating": 5,
      "comment": "An excellent book for mastering MongoDB!",
      "created_at": "2024-11-17T10:00:00Z"
    },
    {
      "review_id": "review_2",
      "user": {
        "user_id": "user_2",
        "name": "Bob",
        "email": "bob@example.com"
      },
      "rating": 4,
      "comment": "A solid resource, but a bit dense in places.",
      "created_at": "2024-11-16T15:30:00Z"
    }
  ]
}
```

Subset Pattern

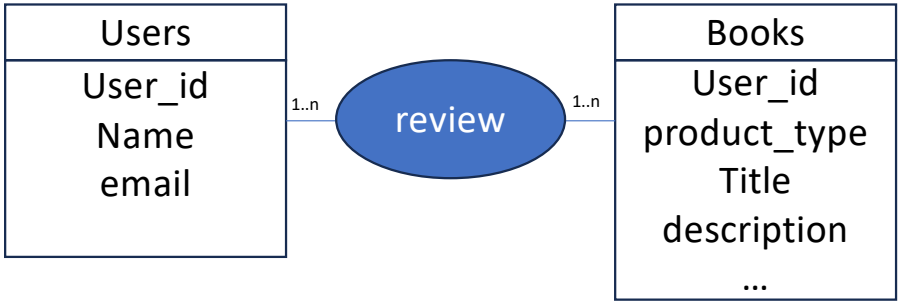


- Le subset pattern consiste conserver que les données utiles dans la collection principale (book) pour les besoins de l'application.
- Les données moins pertinentes sont alors conservées dans une collection indépendante (Reviews)

- On conserve dans un document book uniquement les 10 derniers avis du produit : ceux pertinents pour vos clients.



Single collection Pattern





Si le besoin de l'application est d'afficher l'ensemble des reviews alors le Subset Pattern ne suffira pas.

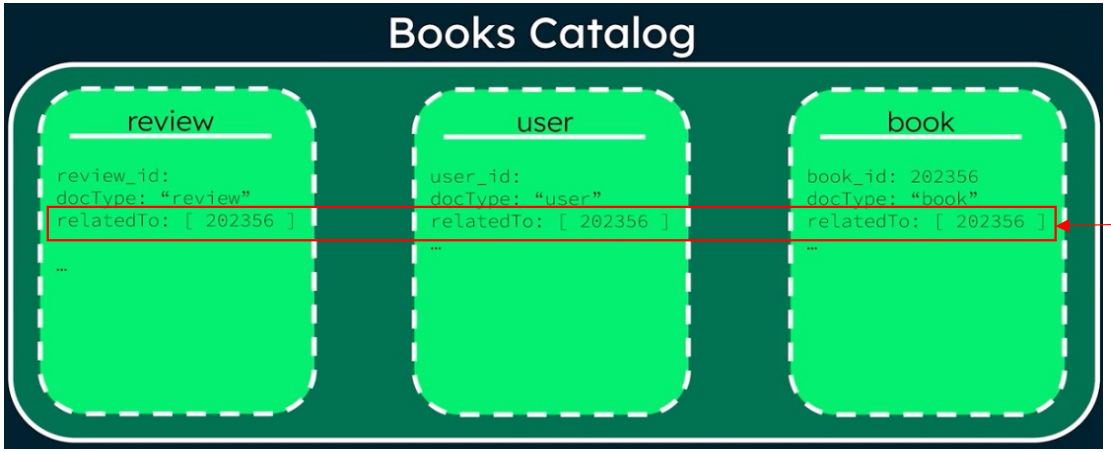
Le choix d'un modèle par référence semble être adapté mais …
au niveau des performances, c'est pas cool !!!



Single collection Pattern

- Basé sur le pattern d'héritage
- On définit une clé :
  - docType : pour identifier le type de document
  - 'relatedTo' pour conserver le lien entre les documents

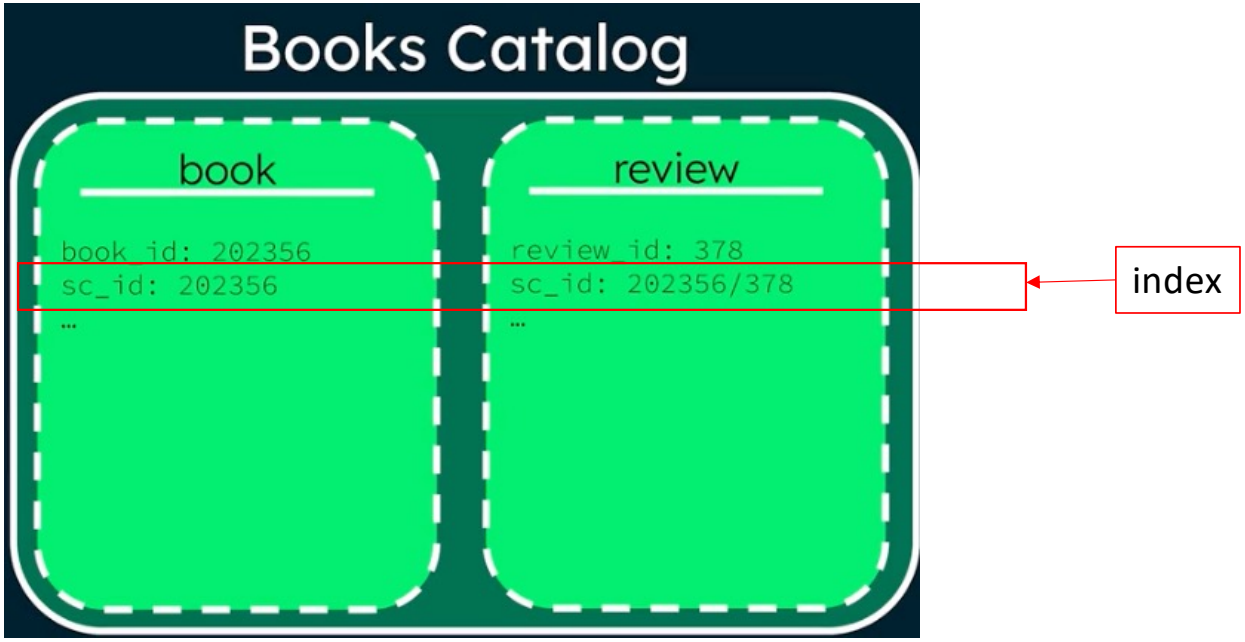




- On regroupe l'ensemble de nos collections dans une seule collection et on crée un index sur la clé qui mettra en évidence les objects en relation (i.e relatedTo)
- Les données qui sont accédées ensembles doivent être stockées ensembles

Single collection Pattern

- Bien entendu, il est possible d'utiliser ce pattern avec les relations 1à N.
- Par exemple, si notre application ne s'intéressait qu'aux reviews par book.



- Comment identifier les reviews d'un libre en particulier ?

```
db.books_catalog.find({"sc_id": {'$regex': "^202356/"}})
```

SINGLE COLLECTION PATTERN

Stocker plusieurs types d'entités et leurs relations dans une seule collection.

Principe : On utilise un champ discriminant (ex: _type) pour identifier le type de chaque document.

1 UNE SEULE COLLECTION

Tous les documents ensemble

Collection : ecommerce

La collection contient des documents de types différents : customers, products, orders, reviews... identifiés par le champ _type.

2 EXEMPLES DE DOCUMENTS DANS LA COLLECTION "ecommerce"

Client

_type : "customer"

```
{
  "_id": "c1",
  "_type": "customer",
  "name": "Alice Martin",
  "email": "alice@mail.com",
  "created_at":
    ISODate("2025-04-26")
}
```

Produit

_type : "product"

```
{
  "_id": "p1",
  "_type": "product",
  "name": "Casque Bluetooth",
  "price": 99.99,
  "category": "Audio",
  "stock": 120
}
```

Commande

_type : "order"

```
{
  "_id": "o1",
  "_type": "order",
  "customer_id": "c1",
  "items": [
    { "product_id": "p1",  "qty": 1,
    ,  "price": 99.99 },
    { "product_id": "p2",  "qty": 2,
    ,  "price": 19.90 }
  },
  "total": 139.79,
  "status": "paid",
  "created_at": ISODate("2025-04-26")
}
```

Avis

_type : "review"

```
{
  "_id": "r1",
  "_type": "review",
  "product_id": "p1",
  "customer_id": "c1",
  "rating": 5,
  "comment": "Excellent son !",
  "created_at":
    ISODate("2025-04-27")
}
```

3 REQUÊTES PAR TYPE

On filtre avec le champ _type

Toutes les commandes d'un client

```
db.ecommerce.find({
  _type: "order",
  customer_id: "c1"
})
```

Tous les produits en stock

```
db.ecommerce.find({
  _type: "product",
  stock: { $gt: 0 }
})
```

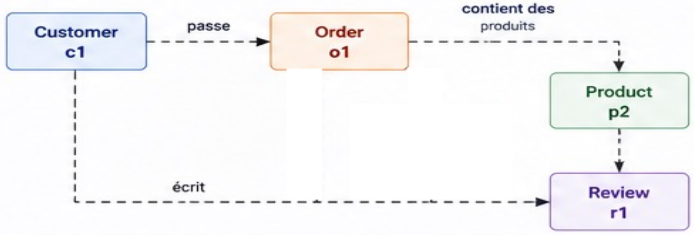
Les avis pour un produit

```
db.ecommerce.find({
  _type: "review",
  product_id: "p1"
})
```

Détail d'une commande

```
db.ecommerce.findOne({
  _type: "order",
  _id: "o1"
})
```

4 RELATIONS ET RÉFÉRENCES



Toutes les entités et leurs relations sont stockées dans la même collection. Les liens sont assurés par des champs de référence (ex: customer_id, product_id).

AVANTAGES

- Simplicité : une seule collection à gérer
- Pas de jointures (lookups) entre collections
- Requêtes plus rapides (un seul point d'accès)
- Flexibilité : ajout facile de nouveaux types
- Cohérence des données et transactions plus simples

LIMITES

- Documents hétérogènes (structures différentes)
- Indexation à bien concevoir (index sur _type souvent nécessaire)
- Peut devenir volumineux si beaucoup de types et de données
- Moins adapté si les accès sont très spécialisés et isolés par type

CAS D'USAGE TYPIQUES

- E-commerce
- CMS (contenu multi-types)
- Systèmes d'événements (activity feeds)
- Applications avec entités liées

INDEX RECOMMANDÉS

- Index sur _type pour filtrer rapidement par type
- Index composés selon les accès fréquents, ex : { _type: 1, customer_id: 1 }
- Index sur les champs de référence (product_id, customer_id, etc.)

Exemple d'index

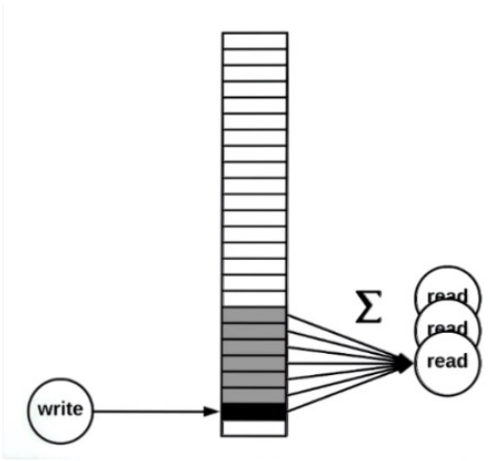
```
db.ecommerce.createIndex({ _type: 1 })
db.ecommerce.createIndex({ _type: 1, customer_id: 1 })
db.ecommerce.createIndex({ _type: 1, product_id: 1 })
```

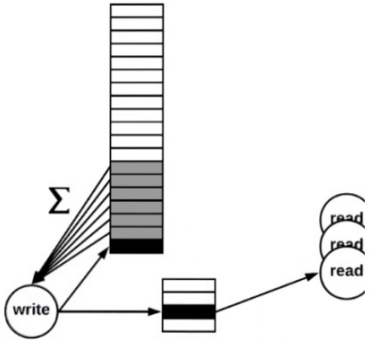
Computing pattern

Stocker des données n'est pas toujours suffisant. Leur utilité se révèle lorsqu'on peut en tirer des calculs, comme le nombre de vues d'un produit ou le nombre de spectateurs d'un film.

Faire ces calculs à chaque requête, surtout sur de grandes quantités de données, consomme beaucoup de ressources (CPU, mémoire, accès disque).

Dans une application qui affiche des informations sur les livres, recalculer constamment le nombre de vues pour chaque visite serait coûteux en termes de ressources.





Il est souvent inutile d'avoir les chiffres exacts en temps réel. Les calculs peuvent être faits en arrière-plan, et les résultats mis à jour périodiquement dans les documents de la base de données pour éviter une surcharge du CPU.

Ce modèle est utilisé lorsque des calculs doivent être effectués de manière répétée dans une application. Il est particulièrement efficace dans des environnements où les lectures sont plus fréquentes que les écritures.

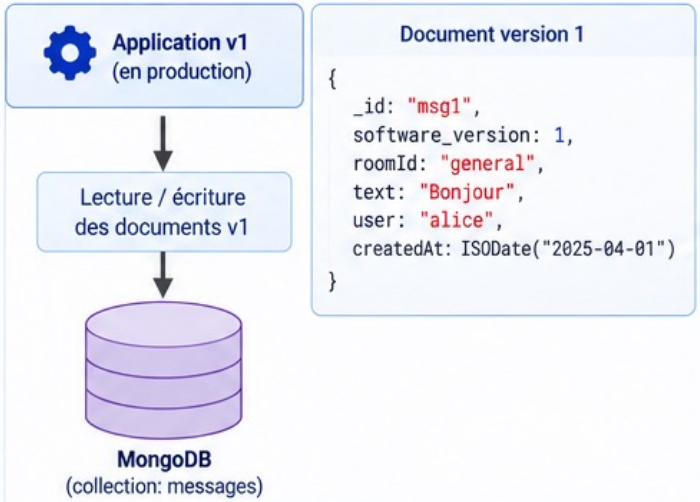
VERSIONING PATTERN

Mettre à jour une application sans l'arrêter même si la structure des documents change.

Principe : chaque document possède une clé **`software_version`** indiquant la version du code applicatif à utiliser.
L'application conserve les anciennes versions du code et gère plusieurs schémas de documents en parallèle.

1 Avant la mise à jour

Application en version 1



Document version 1

```
{
  _id: "msg1",
  software_version: 1,
  roomId: "general",
  text: "Bonjour",
  user: "alice",
  createdAt: ISODate("2025-04-01")
}
```

VERSIONING PATTERN

Mettre à jour une application sans l'arrêter même si la structure des documents change.

Principe : chaque document possède une clé **`software_version`** indiquant la version du code applicatif à utiliser.
L'application conserve les anciennes versions du code et gère plusieurs schémas de documents en parallèle.

1 Avant la mise à jour

Application en version 1

Application v1
(en production)

Lecture / écriture
des documents v1

MongoDB
(collection: messages)

Document version 1

```
{
  _id: "msg1",
  software_version: 1,
  roomId: "general",
  text: "Bonjour",
  user: "alice",
  createdAt: ISODate("2025-04-01")
}
```

2 Déploiement de la nouvelle version

L'application est mise à jour sans arrêt

Application v2
(nouvelle version)

Application v1
(ancienne version
conservée)

L'application contient la logique
pour lire les deux versions
de documents (v1 et v2).

3 Accès à un document

Migration à la volée (lazy migration)

1 Lecture d'un document v1

L'application détecte `software_version: 1`
et utilise l'ancien code pour le lire.

```
{
  _id: "msg1",
  software_version: 1,
  roomId: "general",
  text: "Bonjour",
  user: "alice"
}
```

2 Mise à jour du document

Le document est converti en version 2
et réécrit dans la collection.

```
{
  _id: "msg1",
  software_version: 2,
  roomId: "general",
  content: { text: "Bonjour" },
  author: { id: "alice", name: "Alice" },
  createdAt: ISODate("2025-04-01")
}
```

4 Migration en arrière-plan

Un batch convertit progressivement les documents

Tâche planifiée (batch)
s'exécute périodiquement

Recherche des documents v1

```
db.messages.find({ software_version: 1 })
```

Conversion vers v2

Transformation du schéma

```
{ roomId, text, user } →
{ roomId, content: { text }, author: { ... }}
```

Mise à jour du document

```
db.messages.updateOne(
  { _id: "msg1" },
  { $set: { software_version: 2,... } }
)
```

5 Évolution de la collection dans le temps

État initial : tous les documents en v1

```
_id: msg1
software_version: 1
roomId: "general"
text: "Bonjour"
user: "alice"
```

```
_id: msg2
software_version: 1
roomId: "random"
text: "Hello"
user: "bob"
```

```
_id: msg3
software_version: 1
roomId: "general"
text: "Salut"
user: "carol"
```

...

Utilisation
de l'application
(migration à la volée)

État intermédiaire : mélange v1 et v2

```
_id: msg1
software_version: 2
roomId: "general"
content: { text: "..." }
author: { id: "alice" }
```

```
_id: msg2
software_version: 1
roomId: "random"
text: "Hello"
user: "bob"
```

```
_id: msg3
software_version: 2
roomId: "general"
content: { text: "..." }
author: { id: "carol" }
```

...

Après le batch
(ou au fil du temps)

État final : tous les documents en v2

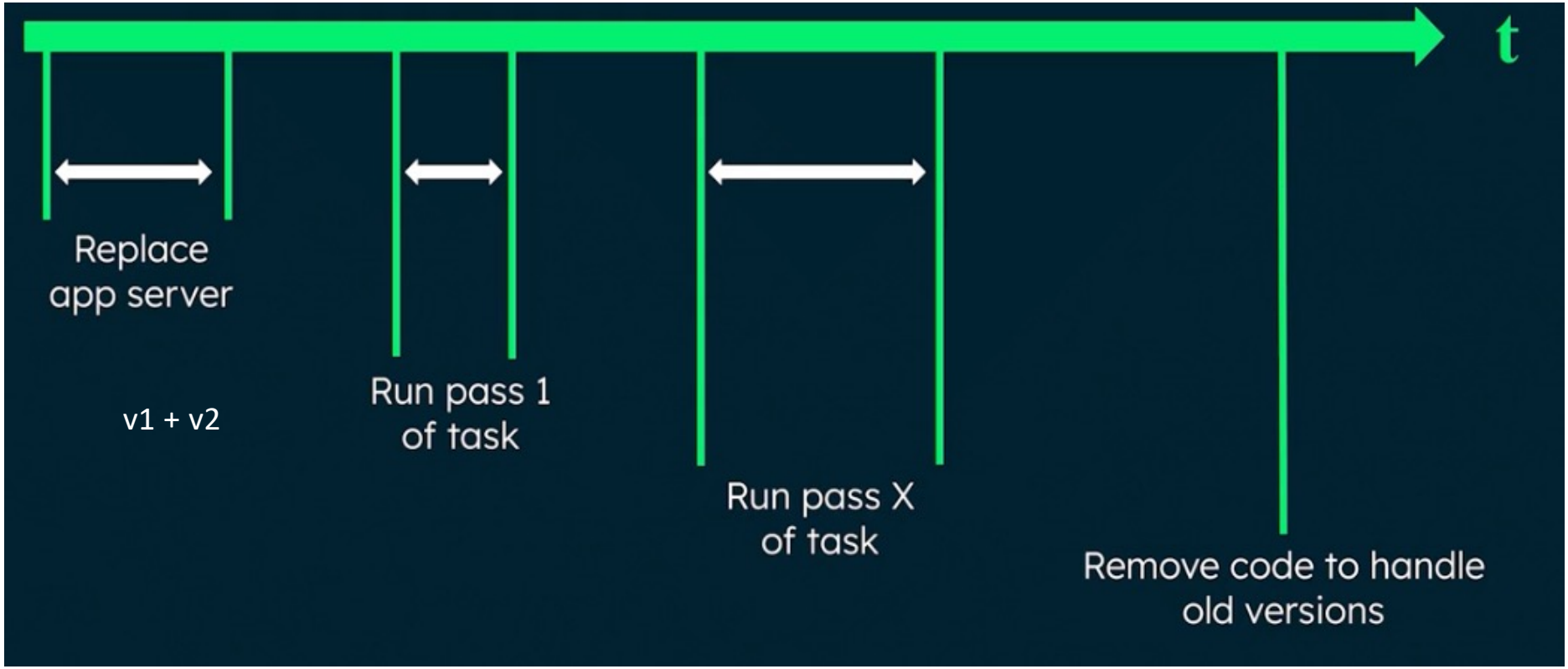
```
_id: msg1
software_version: 2
roomId: "general"
content: { text: "..." }
author: { id: "alice" }
```

```
_id: msg2
software_version: 2
roomId: "random"
content: { text: "..." }
author: { id: "bob" }
```

```
_id: msg3
software_version: 2
roomId: "general"
content: { text: "..." }
author: { id: "carol" }
```

...

Document Versioning pattern



- Une autre approche consiste à mettre à jour les documents lorsqu'ils sont accédés et d'appliquer plus tard des batchs pour mettre à jour les documents restants.

TP


DESIGN PATTERNS
DESIGN PATTERNS EVERYWHERE
makeameme.org